VULNERABILITY ASSESSMENT REPORT

Future Interns - Cyber Security Task 1

MARCH, 2026

Prepared by:
Shuseel Baral

Abstract

This report documents the comprehensive vulnerability assessment conducted on the Damn Vulnerable Web Application (DVWA) as part of the Future Interns Project – Cybersecurity Task 1. The penetration test focused on identifying critical web application vulnerabilities using both manual and automated analysis. Manual analysis was performed with particular emphasis on SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) flaws. Testing was performed on low security levels. Automated analysis was performed using a popular open-source tool, OWSAP ZAP. The assessment revealed severe security weaknesses enabling complete database compromise, authentication bypass, sensitive data exfiltration, and client-side code execution. This report provides detailed technical findings, risk assessments, and prioritized remediation recommendations based on industry best practices and standards.

Keywords: Web Application Security, Penetration Testing, SQL Injection, XSS, CSRF, DVWA, Vulnerability Assessment

General Introduction

This penetration testing engagement was conducted as part of the Future Interns Project – Cybersecurity Task 1, which provides practical experience in web application security assessment using ethical hacking methodologies. The assessment targeted DVWA (Damn Vulnerable Web Application), a purposefully vulnerable platform designed for security education and training. The primary objectives were to identify, exploit, and document common web application vulnerabilities while developing hands-on skills in security testing tools and techniques. The testing scope encompassed SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) vulnerabilities on a low security configuration, intending to evaluate defensive mechanisms and provide actionable remediation guidance. The methodology combined manual testing approaches and automated tools, following established standards including the OWASP Testing Guide and Penetration Testing Execution Standard (PTES). This report presents the findings, analyses, and recommendations derived from this comprehensive security assessment.

Table of Contents

Abstract **1**

General Introduction **2**

Chapter 1: Executive Summary **4**

1.1 Overview 4

1.2 Key Findings 4

1.3 Risk Assessment 4

1.4 Scope of Testing 5

1.5 Target Information 5

Chapter 2: Methodology **6**

2.1 Testing Approach 6

2.2 Tools Used 6

2.3 Testing Standards Compliance 6

Chapter 3: Vulnerability Findings (Manual Analysis) **8**

3.1 SQL Injection 8

3.2 Cross-Site Scripting (XSS) 11

3.3 Cross-Site Request Forgery (CSRF) 16

Chapter 4: Vulnerability Findings (Automated Analysis) **19**

4.1 Port scan using Nmap 19

4.2 Vulnerability scan using OWASP ZAP 19

Chapter 5: Conclusion and Strategic Recommendations **25**

5.1 Explanation of Vulnerabilities (Non-Technical) 25

5.2 Key Findings Summary 25

Chapter 1: Executive Summary

1.1 Overview

This report details the findings of a comprehensive security assessment conducted on the target web application. Multiple critical vulnerabilities were identified, primarily SQL injection flaws, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) vulnerabilities on a low security level.

1.2 Key Findings

- 2 Critical, 1 High, 5 Medium, and 4 Low vulnerabilities were identified across the whole assessment.
- Complete database compromise is possible through multiple vectors.
- Authentication bypass achieved via SQL injection.
- Sensitive data exposure, including password hashes and user information.
- Database structure enumeration successfully executed.

1.3 Risk Assessment

The following is the risk classification:

Vulnerability	Count	Overall Severity
No input sanitization allows direct SQL query manipulation.	3	Critical
No input validation or output encoding implemented	2	Critical
No protection against forged requests.	1	High
Content Security Policy (CSP) Header Not Set	4	Medium
Possible to view the directory listing	4	Medium

SSL is not activated	1	Medium
Hidden file found	1	Medium
Missing Anti-clickjacking Header	2	Medium
Cookie without SameSite Attribute	1	*Low*
In Page Banner Information Leak	2	*Low*
Server Leaks Version Information	4	*Low*
X-Content-Type-Options Header Missing	4	*Low*

Table 1.1: Overall Risk Assessment

1.4 Scope of Testing

This section defines the scope and boundaries of the project.

- Testing Scope: Passive scanning only, no exploitation or attacks should be performed, SQL Injection vulnerabilities across the LOW security level.
- Methodology: Manual penetration testing following OWASP Testing Guide v4.2 and automated scanning using OWASP ZAP.
- Testing Period: March 2026

1.5 Target Information

- Application: DVWA (Damn Vulnerable Web Application)
- IP Address: 127.0.0.1/dvwa/
- Web Server: Apache/2.4.65 (Debian)
- Open Port: 80 (HTTP)

Chapter 2: Methodology

2.1 Testing Approach

Security testing was conducted using a combination of manual techniques and automated tools:

1. Manual testing for SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).
2. Automated scanning with Nmap and OWSAP ZAP.

2.2 Tools Used

- Nmap (Service & Exposure Scan)
- OWASP ZAP (Passive Vulnerability Scan)
- Browser DevTools (Header & Client-Side Analysis)

2.3 Testing Standards Compliance

Testing was conducted in accordance with:

- OWASP Web Security Testing Guide (WSTG)
- PTES (Penetration Testing Execution Standard)
- NIST SP 800-115 Technical Guide to Information Security Testing

2.4 Risk Rating Methodology

Vulnerabilities were rated using the following criteria:

Low	Medium	High	Critical
Vulnerabilities may not have a public exploit (code) available or may not be exploitable in the wild.	Vulnerabilities may not have a public exploit (code) available or may not be exploitable in the wild. Patch/ workaround not yet released by vendor.	Vulnerabilities that can be exploited publicly, workarounds, or fixes/patches available from the vendor.	Vulnerabilities that can be exploited publicly, workarounds, or fixes/patches may not be available from the vendor.

Table 2.1: Risk Rating

2.5 Security Levels

The vulnerability assessment is conducted under the following security level settings available in the application.

Security Level	Description
Low	This security level is completely vulnerable and has no security measures at all.
Medium	This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application.
High	This option is an extension of the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code.

Table 2.2: Security Levels

We can set the security level using the following process.

1. Log in to the DVWA application using the credentials “admin/admin”.
2. Set the security level “Low”, “Medium”, or “High” from the page “DVWA Security”.

Chapter 3: Vulnerability Findings (Manual Analysis)

3.1 SQL Injection

- Vulnerability: No input sanitization allows direct SQL query manipulation.
- Impact: Full database exfiltration (Usernames, Passwords, Schema).
- **Risk Rating: Critical**
- Remediation: Implement proper input validation and sanitization. Use parameterized queries.

A. Boolean-Based SQL Injection

Proof of Concept:

By entering a payload **' OR '1'='1**, an attacker can trick the database into returning all records, effectively bypassing the need for a specific User ID.

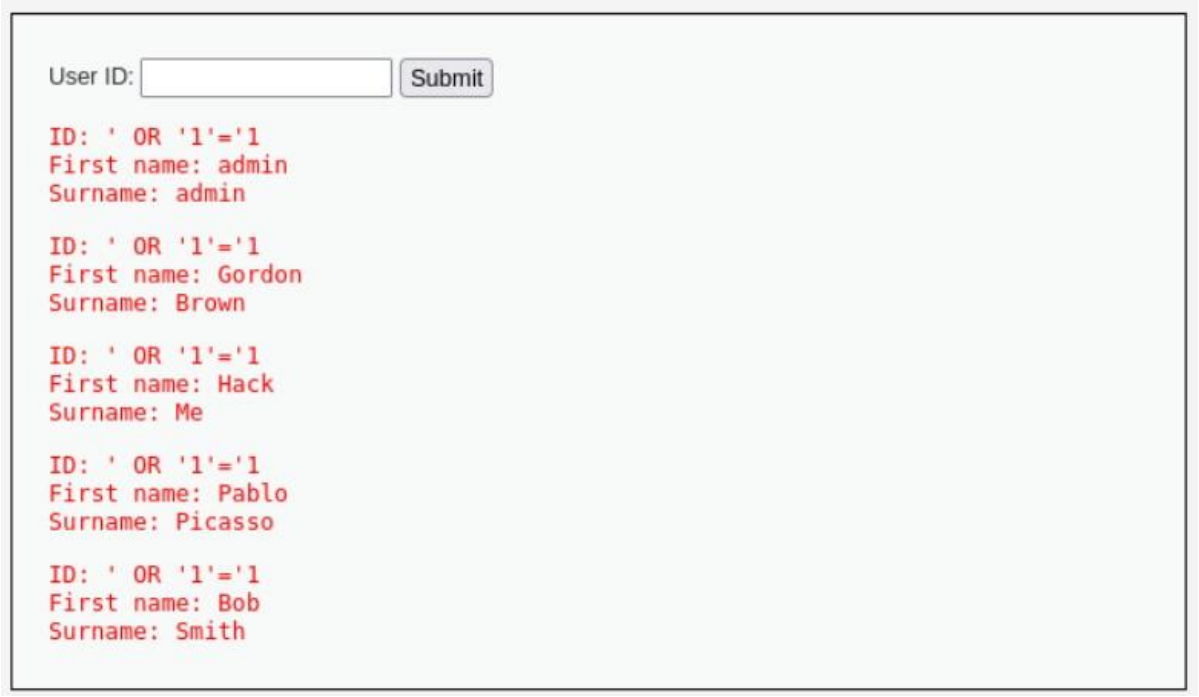

Figure 3.1: Username and Password Disclosure

Technical Details

- Vulnerable Parameter: id
- Target URL: /dvwa/vulnerabilities/sqli/
- Original Query: SELECT first_name, last_name FROM users WHERE user_id = '$id'
- Modified Query: SELECT first_name, last_name FROM users WHERE user_id = '' OR '1'='1'

Impact Analysis

Impact	Severity
Complete authentication bypass	Critical
Access to all user accounts	Critical
Exposure of sensitive user data	High

Table 3.1: Impact Analysis

B. Union-Based SQL Injection

Proof of Concept:

By entering a payload **1' OR 1=1 UNION SELECT DATABASE(), USER()#**, an attacker can extract the database and database user name.

```
ID: 1' OR 1=1 UNION SELECT DATABASE(), USER()#
First name: dvwa
Surname: dvwa@localhost
```

Figure 3.2: Database Name and User Disclosure

Extracted Information:

- Database Name: dvwa
- Database User: dvwa@localhost

By entering a payload **1' OR 1=1 UNION SELECT VERSION(), @@HOSTNAME#**, an attacker can extract the database version and hostname.

```
ID: 1' OR 1=1 UNION SELECT VERSION(), @@HOSTNAME#
First name: 11.8.3-MariaDB-1+b1 from Debian
Surname: kali
```

Figure 3.3: Database Version and Hostname Disclosure

Extracted Information:

- Database Version: 11.8.3-MariaDB-1+b1
- Hostname: kali

By entering a payload **1' OR 1=1 UNION SELECT USER, PASSWORD FROM users #**, an attacker can extract the username and password.

```
ID: 1' OR 1=1 UNION SELECT USER, PASSWORD FROM users #
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' OR 1=1 UNION SELECT USER, PASSWORD FROM users #
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' OR 1=1 UNION SELECT USER, PASSWORD FROM users #
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' OR 1=1 UNION SELECT USER, PASSWORD FROM users #
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' OR 1=1 UNION SELECT USER, PASSWORD FROM users #
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

Figure 3.4: Username and Password Hash Disclosure

Extracted Credentials:

Username	Password Hash (MD5)
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

Table 3.2: Extracted User Credentials

Impact Analysis

Impact	Severity
Username and Password Hash Disclosure	Critical
Database Name and User Disclosure	High
Database Version and Hostname Disclosure	High

Table 3.3: Impact Analysis

3.2 Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) testing was performed on the DVWA (Damn Vulnerable Web Application) platform. Testing covered all three vulnerability levels (Low, Medium, High) across three XSS types: Reflected, Stored, and DOM-based XSS.

- Vulnerability: No input validation or output encoding implemented
- Impact: Session hijacking, malware distribution, phishing & credential theft may occur due to direct injection of JavaScript code into the application.
- **Risk Rating: Critical**
- Remediation: Implement output encoding/escaping to neutralize injected scripts. Validate user-supplied data.

3.2.1 Reflected XSS

With a reflected XSS attack, a malicious script is reflected to the user's browser from a web server, where the script is activated when the victim clicks a link. Reflected XSS will send a request to the web application that has a vulnerability, enabling execution of the malicious script.

Proof of Concept:

Technical Details

- Security level: Low
- Vulnerable Parameter: name
- Target URL: /dvwa/vulnerabilities/xss_r/
- Payload Used: <script>alert("You are hacked!")</script>

Procedure:

An alert pop-up box will appear with the words "You are hacked!" while entering the payload in the "What's your name?" box and clicking Submit.

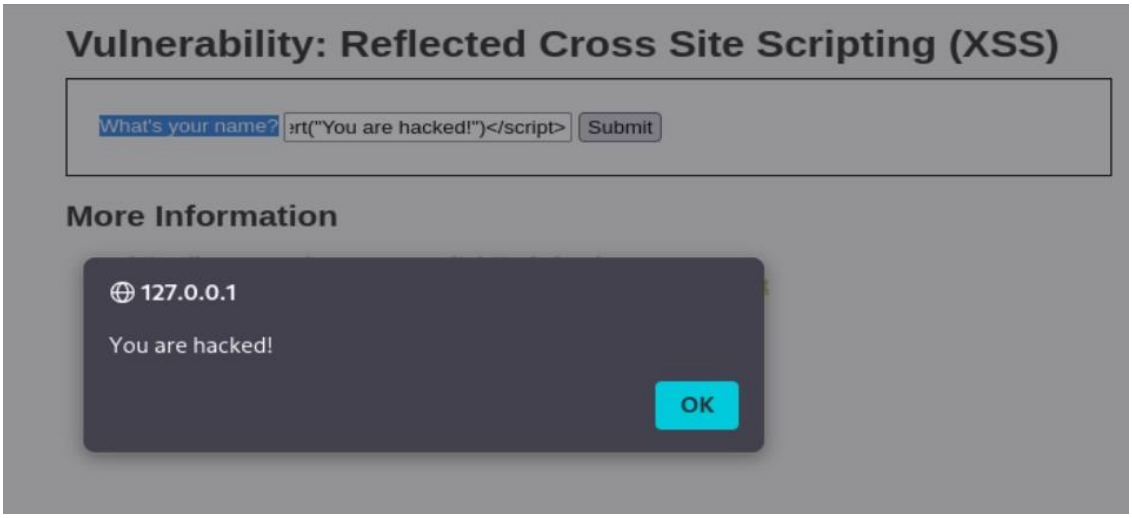

Figure 3.5: Reflected XSS Vulnerability

While viewing the source code of that page, the following lines of code can be observed.

```php
<?php

header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Feedback for end user
    echo '<pre>Hello ' . $_GET[ 'name' ] . '</pre>';
}

?>
```

Figure 3.6: Sample Vulnerable PHP Code

Remediation:

This vulnerability can be remediated using the following sample lines of code.

```php
<?php

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $name = htmlspecialchars( $_GET[ 'name' ] );

    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Figure 3.7: Sample Secured PHP Code

3.2.2 Stored Cross-Site Scripting

In stored cross-site scripting, a malicious script through user input, and the script is stored on the target server in a message forum, database, visitor log, or comment field. When a user visits the target, the server exposes the user to the malicious code.

Proof of Concept:

Technical Details

- Security level: Low
- Vulnerable Parameter: message
- Target URL: /dvwa/vulnerabilities/xss_s/
- Payload Used: <script>alert("You are hacked!")</script>

Procedure:

An alert pop-up box will appear with the words "You are hacked!" while entering the payload in the "Message" box and clicking Sign Guestbook. Since this payload is stored within the page, the pop-up will appear every time the page loads.

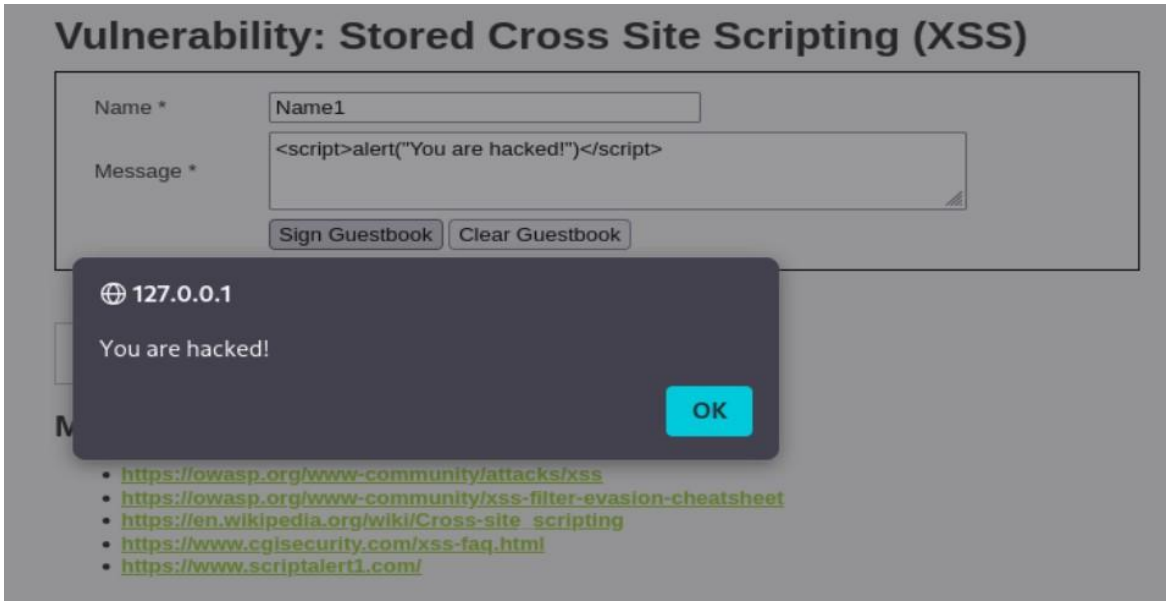

Figure 3.8: Stored XSS Vulnerability

While viewing the source code of that page, the following lines of code can be observed.

```
<?php

if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name    = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],

    // Sanitize name input
    $name = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $

    // Update database
    $query  = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysqli_query($GLOBALS["___mysqli_ston"],  $query ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($GLOBALS[

    //mysql_close();
}

?>
```

Figure 3.8: Sample Vulnerable PHP Code

Remediation:

This vulnerability can be remediated using the following sample lines of code.

```
<?php

if( isset( $_POST[ 'btnSign' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name    = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"]
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = stripslashes( $name );
    $name = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["___mysqli_ston"],
    $name = htmlspecialchars( $name );

    // Update database
    $data = $db->prepare( 'INSERT INTO guestbook ( comment, name ) VALUES ( :message, :name );' );
    $data->bindParam( ':message', $message, PDO::PARAM_STR );
    $data->bindParam( ':name', $name, PDO::PARAM_STR );
    $data->execute();
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Figure 3.9: Sample Secured PHP Code

Recommended rules for preventing XSS attacks

According to OWASP, the following are the general rules for preventing cross-site scripting (XSS) attacks:

- Use an auto-escaping template system.
- Never insert untrusted data except in allowed locations.
- Use HTML escape before inserting untrusted data into HTML element content.
- Use attribute escape before inserting untrusted data into HTML common attributes.
- Use JavaScript escape before inserting untrusted data into JavaScript data values.
- Use CSS escape and strictly validate before inserting untrusted data into HTML-style property values.
- Use URL escape before inserting untrusted data into HTML URL parameter values.
- Sanitize HTML markup with a library such as ESAPI to protect the underlying application.
- Use the HTTPOnly cookie flag.
- Implement content security policy.
- Use the X-XSS-Protection response header.

Reference:
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

3.3 Cross-Site Request Forgery (CSRF)

A CSRF attack can affect applications or websites that rely on a user's identity. In this attack, attackers can trick the user's browser into sending HTTP requests to a target website.

- Vulnerability: No protection against forged requests.
- Impact: An attacker can trick the user into executing unwanted actions, such as changing the password or sensitive data, without user interaction.
- **Risk Rating: High**
- Remediation: Implement anti-CSRF tokens (Synchronizer Token Pattern). Require user interaction for sensitive operations.

Proof of Concept:

Technical Details

- Security level: Low
- Vulnerable Parameter: password
- Target URL: /dvwa/vulnerabilities/csrf/
- Payload Used: ?password_new=newhacked&password_conf=newhacked&Change=Change#

Procedure:

Add the payload after the vulnerable page URL and press Enter. The password will be changed automatically without user interaction.

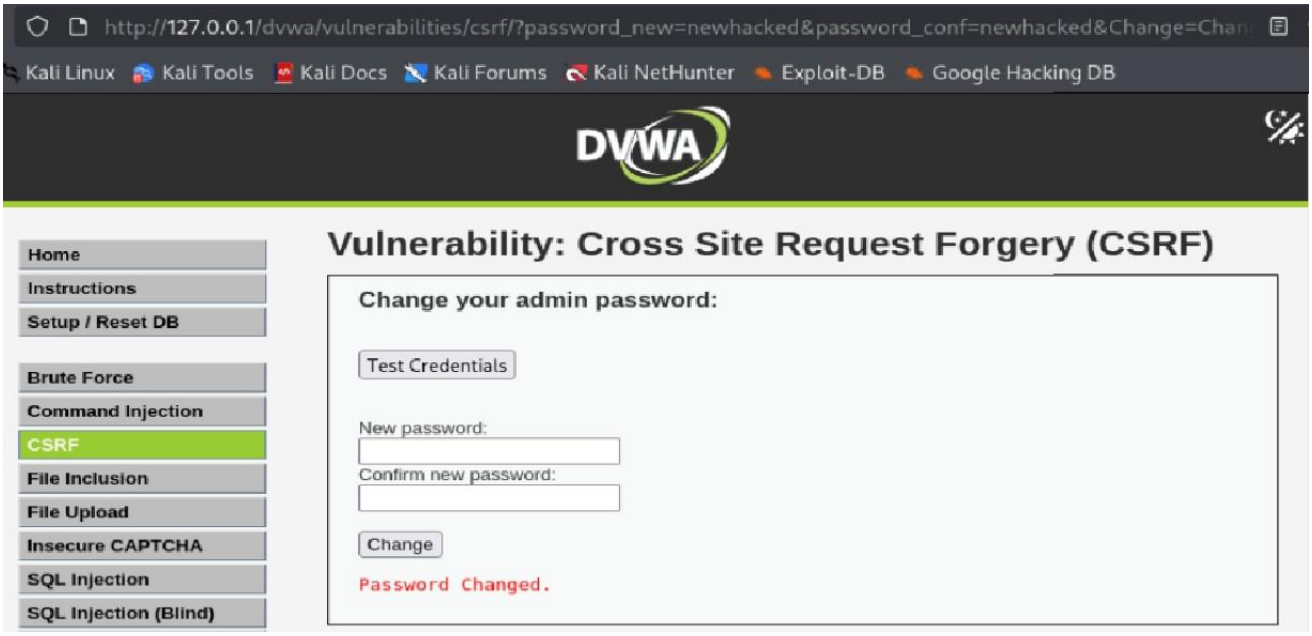

Figure 3.9: Cross-Site Request Forgery (CSRF) Vulnerability

Chapter 4: Vulnerability Findings (Automated Analysis)

4.1 Port scan using Nmap

Nmap ("Network Mapper") is an open-source tool for network exploration and security auditing. It was designed to rapidly scan large networks, although it works fine against single hosts.

- Vulnerability: Server Leaks Version Information.
- Impact: Revealing the server version may help attackers find known vulnerabilities of your web/application server that are subject to.
- **Risk Rating: Low**
- Remediation: Ensure that your web server, application server, load balancer, etc., is configured to suppress the "Server" header or provide generic details.

```
└─$ nmap -sV 127.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-06 07:18 EST
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000010s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
80/tcp   open  http    Apache httpd 2.4.65 ((Debian))
3306/tcp open  mysql?
```

Figure 4.1: Vulnerability scan using Nmap

4.2 Vulnerability scan using OWASP ZAP

The Open Worldwide Application Security Project (OWASP) is a nonprofit foundation that developed the Web Security Testing Guide (WSTG) to test the most common web application security issues.

Vulnerability	Risk Level
Content Security Policy (CSP) Header Not Set	Medium
Possible to view the directory listing	Medium
SSL is not activated	Medium
Hidden file found	Medium
Missing Anti-clickjacking Header	Medium
Cookie without SameSite Attribute	*Low*
In Page Banner Information Leak	*Low*
Server Leaks Version Information	*Low*
X-Content-Type-Options Header Missing	*Low*

Table 4.1: Vulnerabilities found with OWASP ZAP scan

1. Content Security Policy (CSP) Header Not Set

Target URL: /dvwa/, /dvwa/login.php, /robots.txt, /sitemap.xml

Risk Level: **Medium**

Impact: These attacks are used for everything from data theft to site defacement or the distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page. Covered types include JavaScript, CSS, HTML frames, fonts, images, and embeddable objects such as Java applets, ActiveX controls, audio, and video files.

Remediation: Ensure that your web server, application server, load balancer, etc., is configured to set the Content-Security-Policy header.

2. Possible to view the directory listing

Target URL: /dvwa/.git/, /dvwa/dvwa/, /dvwa/dvwa/css/, /dvwa/dvwa/images/

Risk Level: **Medium**

Impact: The directory listing is viewable. A directory listing may reveal hidden scripts, including files, backup source files, etc., which can be accessed to read sensitive information.
Remediation: Disable directory browsing. Make sure the listed files do not induce risks of exposure of sensitive data through directory information.

3. SSL is not activated

Target URL: /dvwa/,
Risk Level: Medium
Impact: The site is only served under HTTP and not HTTPS, so the data transfer is in an unencrypted form. It may risk of a man-in-the-middle attack and other security-related risks.
Remediation: Configure your web or application server to use SSL (https).

4. Hidden file found

Target URL: /dvwa/.git/config
Risk Level: Medium
Impact: Accessing sensitive files may leak administrative, configuration, or credential information, which can be leveraged by a malicious individual to further attack the system or conduct social engineering efforts.
Remediation: Consider whether the component is actually required in production; if it isn't, disable it. If so, ensure that access to it requires appropriate authentication and authorization, or limit exposure to internal systems or specific source IPs, etc.

5. Missing Anti-clickjacking Header

Target URL: /dvwa/, /dvwa/login.php
Risk Level: Medium
Impact: The response does not protect against 'ClickJacking' attacks. It may lead to significant financial theft, credential harvesting, malware installation,

unauthorized camera/microphone access, and severe damage to user trust and brand reputation.
Remediation: The header should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

6. Cookie without SameSite Attribute

Target URL: /dvwa/
Risk Level: Low
Impact: A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. If no SameSite attribute is used may lead to cross-site request forgery, cross-site script inclusion, and timing attacks.
Remediation: Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.

7. In Page Banner Information Leak

Target URL: /robots.txt, /sitemap.xml
Risk Level: Low
Impact: The server returned a version banner string in the response content. Such information leaks may allow attackers to further target specific issues impacting the product and version in use.
Remediation: Configure the server to prevent such information leaks.

8. Server Leaks Version Information

Target URL: /dvwa/, /dvwa/dvwa/css/login.css, /dvwa/login.php, /robots.txt
Risk Level: Low
Impact: The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate

attackers in identifying other vulnerabilities your web/application server is subject to.

Remediation: Ensure that your web server, application server, load balancer, etc., is configured to suppress the "Server" header or provide generic details.

9. X-Content-Type-Options Header Missing

Target URL: /dvwa/, /dvwa/login.php, /dvwa/css/login.css, /dvwa/dvwa/images/login_logo.png

Risk Level: Low

Impact: The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type.

Remediation: Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.

Proof of Concept:

Scanning the application:

Perform a vulnerability scan of the root URL “http://127.0.0.1/dvwa/” of the DVWA application.

Figure 4.2: Automated scan using ZAP

Output of the Scan:

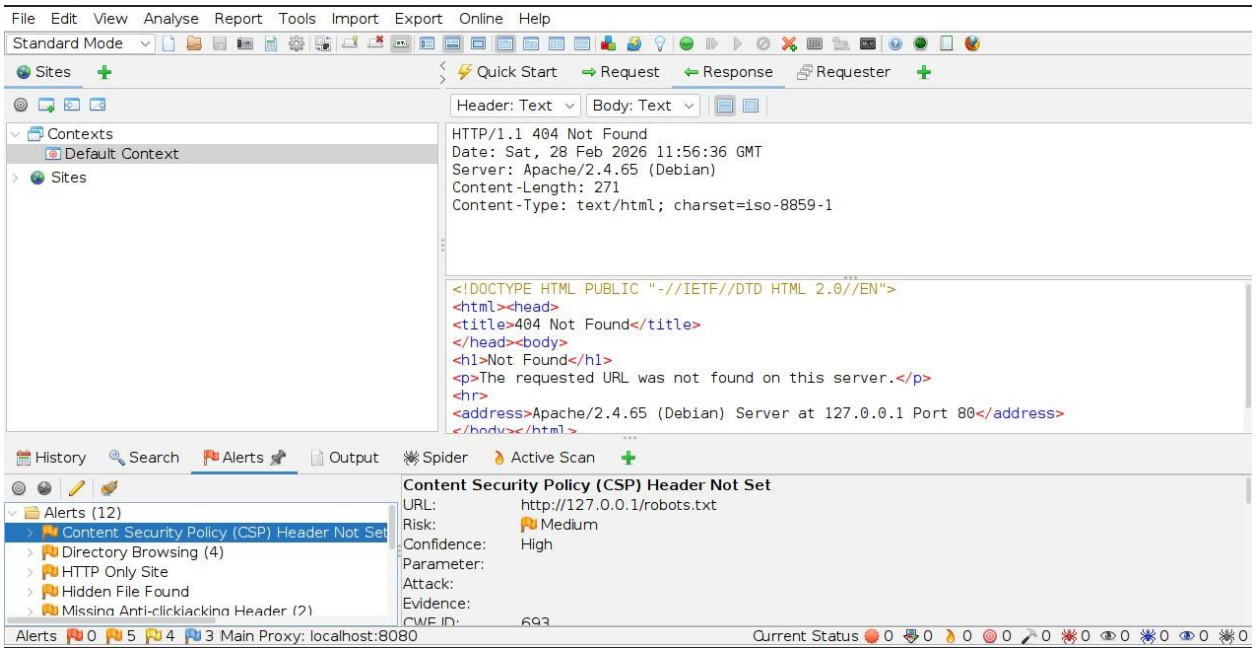

Figure 4.3: Vulnerability scan result on ZAP

Chapter 5: Conclusion and Strategic Recommendations

This vulnerability assessment identified several key weaknesses in the DVWA application. Addressing these vulnerabilities through the recommended remediation steps is crucial to improving the overall security posture of web applications. Further testing and continuous monitoring are recommended to ensure ongoing protection against evolving threats.

5.1 Explanation of Vulnerabilities (Non-Technical)

- **SQL Injection:** An attacker can trick the website into running malicious database commands, potentially stealing or modifying data.
- **Cross-Site Scripting (XSS):** An attacker can inject malicious code that runs in the user's browser, potentially stealing cookies or redirecting the user to a fake website.
- **Cross-Site Request Forgery (CSRF):** An attacker can trick a user into performing actions they didn't intend to, like changing their password or making a purchase.
- **Content Security Policy (CSP) Header:** Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross-Site Scripting (XSS) and data injection attacks.
- **Anti-clickjacking Header:** An anti-clickjacking header (primarily X-Frame-Options) is an HTTP response header that prevents attackers from embedding your website inside an <iframe> on a malicious site. It mitigates UI redress attacks by telling browsers whether to allow rendering of <frame>, <iframe>, or <object> elements.

5.2 Key Findings Summary

Vulnerability	Risk Level	Remediation Priority	Timeframe
SQL Injection	CRITICAL	P1 - Immediate	0-7 days
Reflected XSS	HIGH	P1 - Immediate	0-7 days
CSRF	HIGH	P2 - Urgent	7-14 days
Stored XSS	MEDIUM-HIGH	P2 - Urgent	7-14 days
Information Disclosure	MEDIUM	P3 - High	14-30 days

Table 5.1: Critical Vulnerabilities Overview

5.3 Strategic Recommendations

Immediate (0-30 Days)

- Patch critical SQL injection and XSS vulnerabilities
- Implement Web Application Firewall (WAF)
- Conduct developer security training

Short-term (1-3 Months)

- Deploy Security Development Lifecycle (SDL)
- Implement automated security testing
- Establish security monitoring and alerting

Long-term (3-12 Months)

- Achieve security certification (ISO 27001/PCI DSS)
- Implement bug bounty program
- Develop incident response playbooks